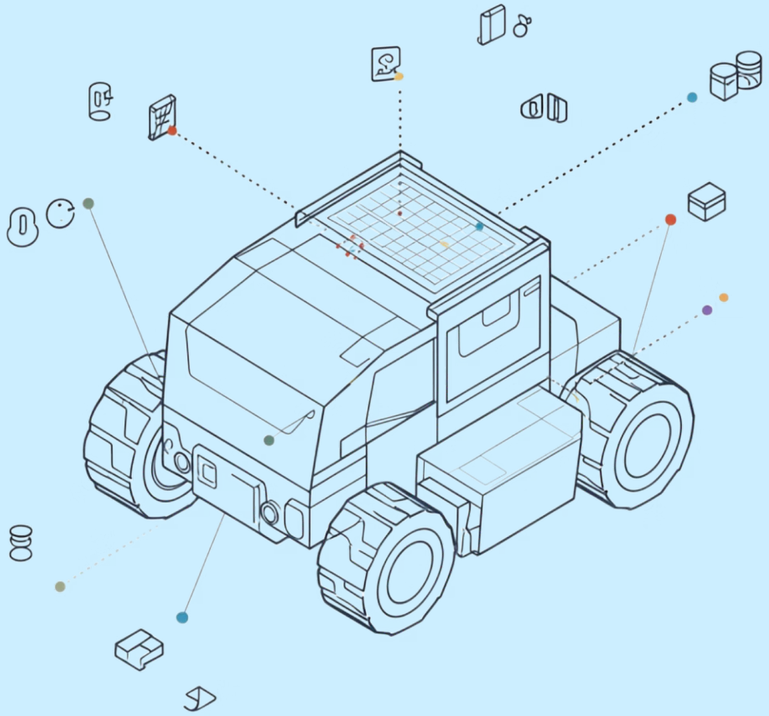
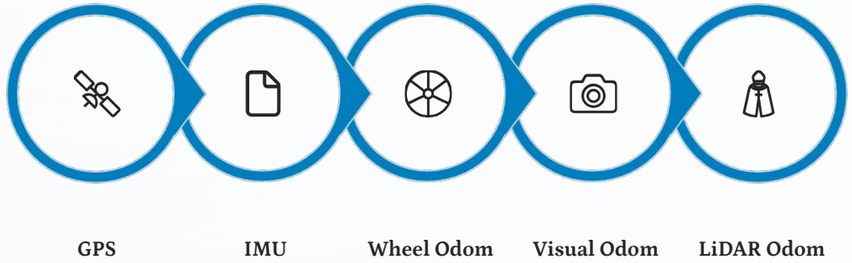


Multi-Sensor Fusion for Mobile Robot Localisation Using Extended Kalman Filter

Dual-EKF Architecture · GPS · IMU · Wheel Odometry · Visual Odometry · LiDAR Odometry

ANISH KUMAR
SACHIT BANSAL
SOMANSHU JUNEJA



5 Sensor Modalities
GPS, IMU, Wheel, Visual, LiDAR

Dual-EKF Architecture
Local + Global filter cascade

Data-Driven Noise Tuning
R matrices from stationary logs

Gazebo Ground Truth
Husarion Lynx UGV · ROS 2 Humble

Why Localisation is Hard

Goal: the robot must estimate its pose **(x, y, θ)** in the world at all times — using sensors that each fail in different ways.

GPS — Global but Noisy

σ ≈ 0.5 m position noise · 10 Hz only · No indoor use · No heading information

IMU — Fast but Drifts

150 Hz · Accelerometer drift grows as $O(t^2)$ · Bias accumulates over time

Wheel Odometry — Simple but Slips

Low noise (σ ≈ 0.03 m/s) · Wheel slip causes unbounded drift · No absolute reference

Visual Odometry — Good Geometry but Jumps

Geometry-consistent · Loop-closure discontinuities · σ ≈ 0.48 m/s vx noise

Solution: Sensor Fusion. Combine all sensors, weighting by reliability. The EKF maintains a probability distribution over pose and updates it as each sensor measurement arrives.

Extended Kalman Filter — Core Idea

The EKF maintains a Gaussian belief **N(x̂, P)** over the robot state, cycling between two steps: **Predict** (uncertainty grows) and **Update** (uncertainty shrinks).

① PREDICT

State prediction:

$$\hat{x}_t^- = f(\hat{x}_{t-1}, u_t)$$

Covariance prediction:

$$P_t^- = F_t P_{t-1} F_t^\top + Q$$

Uncertainty GROWS between measurements. Process noise Q drives the expansion.

② UPDATE

Innovation: *\nu_t = z_t - h(\hat{x}_t^-)*

Kalman Gain:

$$K_t = P_t^- H_t (H_t P_t^- H_t + R)^{-1}$$

State update: *\hat{x}_t = \hat{x}_t^- + K_t \nu_t*

Uncertainty SHRINKS as measurements correct the prediction.

```
15-D State Vector (robot_localization):
x = [ x, y, z, roll, pitch, yaw, vx, vy, vz, vroll, vpitch, vyaw, ax, ay, az ]ᵀ
With two_d_mode: true → effective 6-D: [ x, y, yaw, vx, vyaw, ax ]
```

Predict Step — Motion Model Propagation

Between measurements, the filter propagates state and **grows uncertainty** via the linearised motion model.

Equations

State prediction: $\hat{x}_t^- = f(\hat{x}_{t-1}, u_t)$

Covariance prediction: $P_t^- = F_t P_{t-1} F_t^\top + Q$

Jacobian (linearisation): $F_t = \partial f / \partial x \big|_{\hat{x}_{t-1}}$

Control input `u_t` = `/cmd_vel` (vx, vyaw).

Process noise **Q** encodes uncertainty growth per timestep.

Large Q → fast growth → filter relies more on measurements. Small Q → slow growth → filter trusts its own prediction.

Uncertainty Grows Overtime

At t = 0, Small Ellipse - (Confident)

At t + Delta t, predict only - Ellipse Grows

At t + 2 * Delta t, Measurement - Ellipse Shrinks

Update Step — Measurement Correction

When a sensor reading arrives, the **Kalman Gain** optimally blends prediction and measurement — weighted by their respective uncertainties.

Equations

Innovation: $\nu_t = z_t - h(\hat{x}_t^-)$

Innovation covariance: $S_t = H_t P_t^- H_t^\top + R$

Kalman Gain: $K_t = P_t^- H_t^\top S_t^{-1}$

State update: $\hat{x}_t = \hat{x}_t^- + K_t \nu_t$

Covariance update: $P_t = (I - K_t H_t) P_t^-$

K → 0 if R large (noisy sensor) → trust prediction. **K → 1** if P large (uncertain model) → trust measurement.

Predicted State (wider = uncertain)

Represents the prior estimate before measurement correction, with greater uncertainty shown by a wider spread.

Measurement (narrower = known R)

The sensor observation is more precise when measurement noise R is small, so the distribution appears narrower.

Fused result (narrowest, between both)

The corrected estimate combines both sources and typically lands between prediction and measurement with reduced uncertainty.

Linearisation — Why "Extended" Kalman Filter

Robot motion is **nonlinear**: a Gaussian input through a nonlinear function does *not* produce a Gaussian output.

The EKF linearises via a first-order Taylor expansion to preserve the Gaussian form.

⚠️ Problem: True Nonlinear Transform

Input θ ~ $N(\hat{\theta}, \sigma^2)$ · Output: $v_x \cos(\theta)$

Result: **skewed, non-Gaussian** asymmetric distribution — Kalman maths breaks down entirely.

✓ EKF Fix: First-Order Taylor Expansion

$$f(x) \approx f(\hat{x}) + F(x - \hat{x}), \quad F = \left.\frac{\partial f}{\partial x}\right|_{x=\hat{x}}$$

Tangent line at $\hat{\theta} = 45^\circ$: slope = $-v_x \sin(\hat{\theta})$. Linear approximation → Gaussian output preserved ✓

⚠ Valid only when uncertainty is **small** — the input Gaussian must remain within the linear region. High angular velocity can cause divergence.

EKF Assumptions & When They Break

Understanding the limitations of the linearised Gaussian approximation is essential for robust deployment.

Gaussian Noise

Assumption: w ~ N(0, Q), v ~ N(0, R)

Reality: GPS has non-Gaussian outliers from multipath reflections. Handled via Mahalanobis distance rejection threshold.

Locally Linear Dynamics

Assumption: Nonlinearity mild enough for 1st-order Taylor expansion

Reality: Fails at high angular velocity or sharp turns — EKF can diverge under strong nonlinearity.

Known Noise Covariances

Assumption: Q and R are accurately modelled

Handled ✓ R values measured from real stationary sensor logs. Data-driven tuning is a core contribution of this project.

Independent Noise

Assumption: Process and measurement noise uncorrelated across timesteps

Reality: Wheel slip introduces correlated error. Mitigated by fusing velocity, not position, from wheel odometry.

Five Sensor Modalities — What Each Provides

Each sensor contributes where it is strongest. Noise parameters were measured from **60-second stationary logs** in Gazebo.

Sensor	ROS Topic	Rate	Measures	Noise Characteristics	Role in EKF
IMU (9-DOF)	/imu/data	150 Hz	Accel (ax,ay,az) Gyro (gx,gy,gz)	Accel σ ≈ 0.98 m/s² · Gyro σ ≈ 0.015 rad/s · Bias drift	Short-term orientation dead-reckoning
GPS (NavSat)	/gps/fix	10 Hz	Lat, Lon, Alt	Position σ ≈ 0.5 m · Outlier spikes · No heading	Global position anchor
Wheel Odometry	/diff_drive_controller/odom	38.6 Hz	Pose (integrated) Twist (vx, vyaw)	Low σ ≈ 0.03 m/s vx · Slip accumulates	Primary velocity source
Visual Odom (RTAB-Map)	/odometry/visual	14.3 Hz	Pose from camera features	σ ≈ 0.48 m/s vx · Loop-closure jumps	Geometry-consistent relative motion
LiDAR Odom (rtab ICP)	/odometry/lidar	3.9 Hz	Pose from point cloud	σ ≈ 0.29 m/s vx · Low angular noise	Most reliable geometry · Best loop closure

IMU — Inertial Measurement Unit

Fast, noisy, drifts — but essential for orientation. The IMU runs at **150 Hz**, providing the highest-frequency input to the filter.

Physical Principle & ROS Configuration

Accelerometer: measures specific force (accel − gravity). Double integration gives position — noise accumulates as $O(t^2)$, quadratic drift.

Gyroscope: measures angular rate. Single integration gives orientation. Drift is $O(t)$ — linear.

`imu0_remove_gravitational_acceleration: true` — subtracts 9.81 m/s² from az before fusion

`imu0_differential: true` — no magnetometer in simulation → only change in angle is fused, eliminating the 150° heading runaway seen in early experiments

Orientation rows (4–6) **DISABLED** — fusing both orientation and angular velocity would double-count the biased gyro signal

Measured Noise — Stationary 60 s

```
Accel X var=0.969 σ=0.985 m/s²
Accel Y var=0.565 σ=0.752 m/s²
Accel Z var=0.691 σ=0.831 m/s²
Gyro X var=0.000226 σ=0.015 rad/s
Gyro Y var=0.001462 σ=0.038 rad/s
Gyro Z var=0.0416 σ=0.204 rad/s
```

Key insight: Gyro Z variance (0.0416) is 184× larger than Gyro X (0.000226)

Yaw is the weakest IMU channel

Wheel Odometry — Dead Reckoning from Encoders

Reliable, high-rate velocity source. The critical design decision: fuse **velocity**, not absolute position — preventing wheel slip drift from contaminating the global estimate.

Differential Drive Kinematics

```
# Forward arc and heading change
ΔS  = (ΔS_right + ΔS_left) / 2
Δθ  = (ΔS_right - ΔS_left) / wheel_base
x   = x + ΔS · cos(θ + Δθ/2)
y   = y + ΔS · sin(θ + Δθ/2)
θ   = θ + Δθ
```

Why fuse velocity, not position? Fusing absolute wheel odom position couples the EKF to accumulated slip drift. Fusing only vx and vyaw lets GPS independently correct global position — no drift coupling.

Measured: vx σ ≈ **0.03 m/s** — very clean signal in Gazebo simulation.

EKF Config (odom0)

```
odom0_config:
  [false, false, false,  # x, y, z:    DISABLED
   false, false, false,  # roll,pitch,yaw: OFF
   true,  false, false,  # vx: ENABLED
   false, false, true,   # vyaw: ENABLED
   false, false, false]  # accel: OFF

# vy = 0 mechanically (diff drive)
# cannot move side ways
```

`odom0_relative`: false → measurements in global odom frame

`odom0_differential`: false → wheel odom is stable (no loop closure)

Noise: std_dev ≈ 0.03 m/s → low covariance in YAML config

Visual Odometry & LiDAR Odometry

Geometry-consistent odometry sources with loop-closure discontinuities — requires differential mode for stable EKF integration.

VISUAL ODOMETRY — RTAB-MAP

- Real-time feature matching across frames; homography / essential matrix for camera motion

Loop-closure detection corrects drift via a **sudden absolute pose jump (+2 m)** → EKF diverges

Fix: `odom1_differential: true` → EKF computes Δpose; large jumps rejected by `pose_rejection_threshold: 1.5` (Mahalanobis)

Noise: σ ≈ 0.48 m/s for vx (starburst interference on agricultural tiles); covariance set to **0.5**

LIDAR ODOMETRY — rtab ICP

- Iterative Error State KF (IESIKF) — tightly coupled IMU + LiDAR
- ICP-variant: registers 3D point cloud against local map; no visual features → works in low-texture environments

Loop-closure discontinuities still present → same `differential: true` treatment

vx σ ≈ 0.29 m/s; vyaw σ ≈ 0.21 rad/s; covariance: **0.02** (more trusted than VO)

`pose_rejection_threshold: 2.0` — cleaner signal allows more measurements through

GPS Integration — From Lat/Lon to Local XY

`navsat_transform_node` bridges WGS84 coordinates and the EKF's metric frame.

NavSat Transform Algorithm

01

Wait 3.0 s

Allow EKF to stabilise heading before accepting GPS fixes.

02

Record Origin Datum

First GPS fix becomes the local coordinate origin (lat_o, lon_o).

03

Project to Metric XY

$x = (lat - lat_o) \times 111320$ | $y = (lon - lon_o) \times 111320 \times \cos(lat_o)$

04

Rotate into EKF Frame

Apply IMU heading at initialisation; output `/odometry/gps` → fused into EKF #2.

05

No GPS Heading

At low speed, two consecutive positions have noise-to-signal too high. Heading comes from IMU only.

No GPS heading: at low speed, two consecutive positions have noise-to-signal too high. Heading comes from IMU only.

Measured GPS Noise — Stationary 60 s

Axis	Variance	Std Dev	Notes
Latitude	0.263 m²	0.51 m	Satellite drift + multipath
Longitude	0.245 m²	0.49 m	Slightly better than lat
Altitude	1.230 m²	1.11 m	Worst axis (vertical geometry)

⚠ Random walk radius: ~1 m over a 60-second window.

EKF config: only x,y fused from GPS

```
odom1_config:
 [true, true, false, # x, y: ENABLED
 false, false, false, # orientation: DISABLED
 false, false, false, # (GPS has no heading)
 false, false, false,
 false, false, false]
```

Dual EKF Architecture — Why One EKF Is Not Enough

Separates smooth local estimation from globally-bounded GPS correction.

✗ Single EKF + GPS

- GPS at 10 Hz, σ ≈ 0.5 m
- Growing uncertainty between updates
- GPS arrival → EKF snaps toward fix

Result: visible jitter — downstream planners receive jerky poses

✗ Single EKF without GPS

Wheel + IMU sources only

Drift accumulates **unboundedly**

- No global correction mechanism
- Error grows linearly with distance (confirmed in baseline tests)

✓ Dual EKF Solution

- **EKF #1 - Local (odom frame)**
- Fuses: Wheel + VO + LiDAR + IMU
- No GPS → smooth, continuous

Publishes: `odom→base_link` TF

Output: `/odometry/local`

- **EKF #2 - Global (map frame)**
- Fuses: All above + GPS
- GPS → globally bounded drift

Publishes: `map→odom` TF

Output: `/odometry/global`

How the Two EKFs Interact

- EKF #1 provides smooth high-frequency estimate (50 Hz) for local navigation.

EKF #2 computes the `map→odom` correction offset at ~10 Hz (GPS-driven).

GPS correction in EKF #2 shifts the local frame — **NO jitter reaches EKF #1**.

Global pose = (map→odom from EKF #2) ◦ (odom→base_link from EKF #1). EKF #2 shifts the local frame at ~10 Hz — EKF #1 remains smooth and unaffected.

ROS2 Node Graph — The Local-Global Feedback Loop

How `navsat_transform_node` couples EKF #1 and EKF #2 into a closed feedback architecture.

Node Topology

```
/imu/data ──
```

Sensor Noise Characterisation — How R Values Were Derived

Stationary robot recorded for 60 s; any non-zero variance is pure sensor noise, directly used to populate measurement covariance matrix R.

Channel	Mean	Variance	Std Dev	Notes
GPS Latitude	~0	0.263 m^2	0.51 m	Random walk + satellite drift
GPS Longitude	~0	0.245 m^2	0.49 m	Slightly better than lat
GPS Altitude	~0	1.230 m^2	1.11 m	Worst GPS axis (vertical geometry)
IMU Gyro X	~0	0.000226 $(rad/s)^2$	0.015 rad/s	Clean, good axis
IMU Gyro Y	~0	0.001462 $(rad/s)^2$	0.038 rad/s	Moderate
IMU Gyro Z	~0	**0.041601 $(rad/s)^2$**	**0.204 rad/s**	⚠ Highest — yaw drift
IMU Accel X	~0	0.969666 $(m/s^2)^2$	0.985 m/s^2	Very high — vibration
IMU Accel Y	~0	0.565310 $(m/s^2)^2$	0.752 m/s^2	High
IMU Accel Z	~9.81	0.691300 $(m/s^2)^2$	0.831 m/s^2	Gravity ± noise
Wheel Odom vx	~0	~0.001 $(m/s)^2$	0.032 m/s	Very clean (Gazebo)

⚠ **IMU Gyro Z variance (0.0416) is 184× larger than Gyro X (0.000226).** Yaw estimation from raw IMU is the weakest channel → LiDAR odometry is the primary yaw source.

Sensor Configuration Vectors — What Each `true`/`false` Means

A 15-element boolean vector selects which state dimensions each sensor updates in the EKF.

```
# Index:  0    1    2     3     4    5    6    7    8    9    10    11   12   13   14
# State:  x    y    z  roll  pitch  yaw  vx   vy   vz  vroll vpitch vyaw  ax   ay   az
```

Wheel Odometry (odom0)

```
[false,false,false, # x,y,z: DISABLED — abs pos drifts
 false,false,false, # roll,pitch,yaw: no heading
 true, false,false, # vx: ENABLED — forward velocity
 false,false,true,  # vyaw: ENABLED — turn rate
 false,false,false] # accel: DISABLED
```

vy = 0 mechanically — differential drive cannot move sideways.

IMU (imu0)

```
[false,false,false, # position: none
 false,false,false, # orientation: DISABLED
 false,false,false, # velocity: not measured
 true, true, true,  # vroll,vpitch,vyaw: ENABLED
 true, true, true]  # ax,ay,az: ENABLED
```

Orientation DISABLED — no magnetometer → 150° drift observed when rows were enabled.

LiDAR Odom (odom2)

```
[true, true, false, # x,y: ENABLED — scan match
 false,false,true,  # yaw: ENABLED — scan heading
 false,false,false, # velocities: DISABLED
 false,false,false,
 false,false,false] # differential: true
```

`differential: true` — use Δpose, not absolute pose. Prevents loop-closure jumps.

Process Noise Covariance Q — Tuning Uncertainty Growth

Q controls how fast the filter distrusts its own prediction between measurements.

Q Tuning Principle

High Q → uncertainty grows fast → filter trusts next measurement heavily

Low Q → uncertainty grows slowly → filter trusts its own motion model

Q ≈ (expected state change per timestep)² × safety_factor. The 15×15 Q matrix is diagonal — state dimensions assumed independently uncertain.

State	Q Value	Justification	Derivation
x position	0.025	Tightened from 0.05 — LiDAR reliable	Δx at 50 Hz, 0.5 m/s ≈ 0.01 m → Q ≈ $0.01^2 \times 2.5$ = 0.025
y position	0.025	Same as x (symmetric ground robot)	Symmetric treatment
z position	0.06	Looser — ground robot, z not critical	Not safety-critical
yaw	0.08	Gyro Z is noisiest IMU channel (var=0.0416)	Δθ per step ≈ 0.029 rad → Q_yaw ≈ 0.06
vx	0.04	Raised from 0.025 — wheel slip on varied terrain	Accounts for slip uncertainty
vyaw	0.04	Wheel + IMU both contribute	Moderate uncertainty between updates
ax, ay	0.01	Accelerations change slowly on ground robot	Small step-to-step change expected

Initial Estimate Covariance P_0 — Startup Uncertainty

P_0 encodes how certain we are about the robot's state at t = 0. The two EKFs require fundamentally different initialisations.

EKF #1 — Local (odom frame)

```
initial_estimate_covariance:
  diagonal value = 1e-9
```

- Robot starts at known origin (0, 0, 0)

odom frame is **defined relative to start** — initial pose is exact by definition

- Near-zero uncertainty → EKF trusts its initial state fully
- Avoids unnecessary filter relaxation at startup

EKF #2 — Global (map frame)

```
initial_estimate_covariance:
  x, y, vx, vy = 1e-9 m² / (m/s)²
```

- map frame = real-world absolute frame

Before the first GPS fix, **global position is completely unknown**

- Large P_0 forces EKF to trust the first GPS measurement heavily
- Enables fast initial convergence to GPS position

Asymmetric P_0 is intentional: local certainty vs. global ignorance.

Mahalanobis Distance Gates — Outlier Rejection

Adaptive threshold based on current filter uncertainty P — the gate opens when the filter is uncertain and tightens as it converges.

Innovation Gate Formula

$$d_M^2 = (z - H\hat{x})^\top S^{-1}(z - H\hat{x})$$

where $S = H_t P_t^- H_t^\top + R$ (innovation covariance)

P large → gate OPENS

Filter uncertain; more measurements pass through.

P small → gate TIGHTENS

Filter converged; only close measurements accepted.

Measurements inside the 2σ uncertainty ellipse are accepted; those outside are rejected. Ellipse adapts dynamically with P.

Per-Sensor Gate Configuration

Sensor	Threshold	σ Noise	Reason
Visual Odom pose	1.5	0.48 m/s	Noisy — aggressive rejection (~13% under Gaussian)
LiDAR Odom pose	3.0	0.07 m/s	Cleaner signal — allow more through (~5% rejected)
GPS position	3.0	0.5 m	Known noise level — standard gate
IMU pose	0.8	high	Aggressive — raw IMU full noise
IMU twist	0.8	high	Gyro Z large variance
IMU accel	0.8	0.98 m/s²	Accelerometer very noisy

Differential Mode — Relative vs. Absolute Pose Fusion

Isolates loop-closure jumps for rejection without discarding valid relative motion increments.

✗ Problem: Absolute Pose Fusion

- VO publishes absolute poses in its own frame

RTAB-Map loop closure: x = 5.0 m → x = 7.0 m in **one step**

- EKF sees a sudden +2 m jump

Violates smooth-motion assumption → **filter diverges**

✓ Solution: Differential Mode

```
odom1_differential: true
# EKF computes relative transform:
z_k = T(t_{k-1})^{-1} · T(t_k)
# Loop closure: Δx = 2.0 m
→ d_M >> 1.5 → REJECTED
# Normal motion: Δx = 0.01 m
→ ACCEPTED
```

Absolute vs. Differential — Timeline

Mode	t=1	t=2	t=3	t=4	t=5 (closure)	t=6
Absolute x	1.0	2.0	3.0	5.0	7.0	8.0
Δx (diff)	Δ1.0	Δ1.0	Δ1.0	Δ1.0	REJECT	Δ1.0

⚠ **When NOT to use differential:** Wheel odom (already a rate signal) and GPS (removes the global correction benefit).

Control Input Fusion — Using /cmd_vel for Better Prediction

Incorporating commanded velocity into the predict step reduces prediction error

Without control input

Predict step assumes **CONSTANT VELOCITY**. When robot accelerates/decelerates or turns, prediction error grows.

With `use_control: true` — EKF incorporates `/cmd_vel` into prediction:

`x̂_predict = f( x̂, u_cmd_vel )`

Prediction Quality Comparison

Without Control

Predicted trajectory is a straight line regardless of actual curve. Large residual when sensor arrives. Noisy updates.

With Control

Predicted trajectory follows commanded turn. Smaller innovation (residual) when sensor arrives. More stable filter, fewer large corrections.

Configuration

```
use_control: true
control_config: [true, false, false, false, false, true]
                # vx only + vyaw only (diff drive, no vy)

acceleration_limits:  [1.0, 0.0, 0.0, 0.0, 0.0, 1.5]  # max
m/s² and rad/s²
deceleration_limits:  [1.0, 0.0, 0.0, 0.0, 0.0, 1.5]
acceleration_gains:   [0.8, 0.0, 0.0, 0.0, 0.0, 0.9]  #
trust new cmd
deceleration_gains:   [1.0, 0.0, 0.0, 0.0, 0.0, 1.0]
```

Frame Configuration — odom vs. map World Frame

TF hierarchy and `two_d_mode` effects on the state vector.

TF Frame Hierarchy

```
map          (GPS-corrected global)
│   ← EKF #2 publishes map→odom
odom         (local smooth frame)
│   ← EKF #1 publishes
odom→base_link
base_link    (robot body)
├── imu_link
├── gps_link
├── camera_link
└── lidar_link  (/tf_static)
```

EKF #1 — world_frame: odom

Maintains `odom→base_link`. Smooth, continuous at 50 Hz. No GPS — drifts over time, but drift is smooth. Used for local navigation and control.

EKF #2 — world_frame: map

Maintains `map→odom` as a slowly-updated correction offset. GPS shifts this offset to correct global position without disrupting the smooth odom frame.

two_d_mode: true

Forces z=0 (ground robot). Removes roll/pitch from state. Zeroes z, roll, pitch rows in Q, P, and sensor config. Reduces effective state **15D → 6D**. Faster computation, no 3D drift.

Baseline EKF Results — Performance Analysis

Gazebo ground truth comparison: **|ekf_global – ground_truth|** in metres. Husarion Lynx UGV driven in a figure-8 pattern for 2+ minutes. Ground truth from OdometryPublisher plugin (zero noise, exact model pose, 50 Hz).

📈 Trajectory Plot

White dashed: Gazebo ground truth

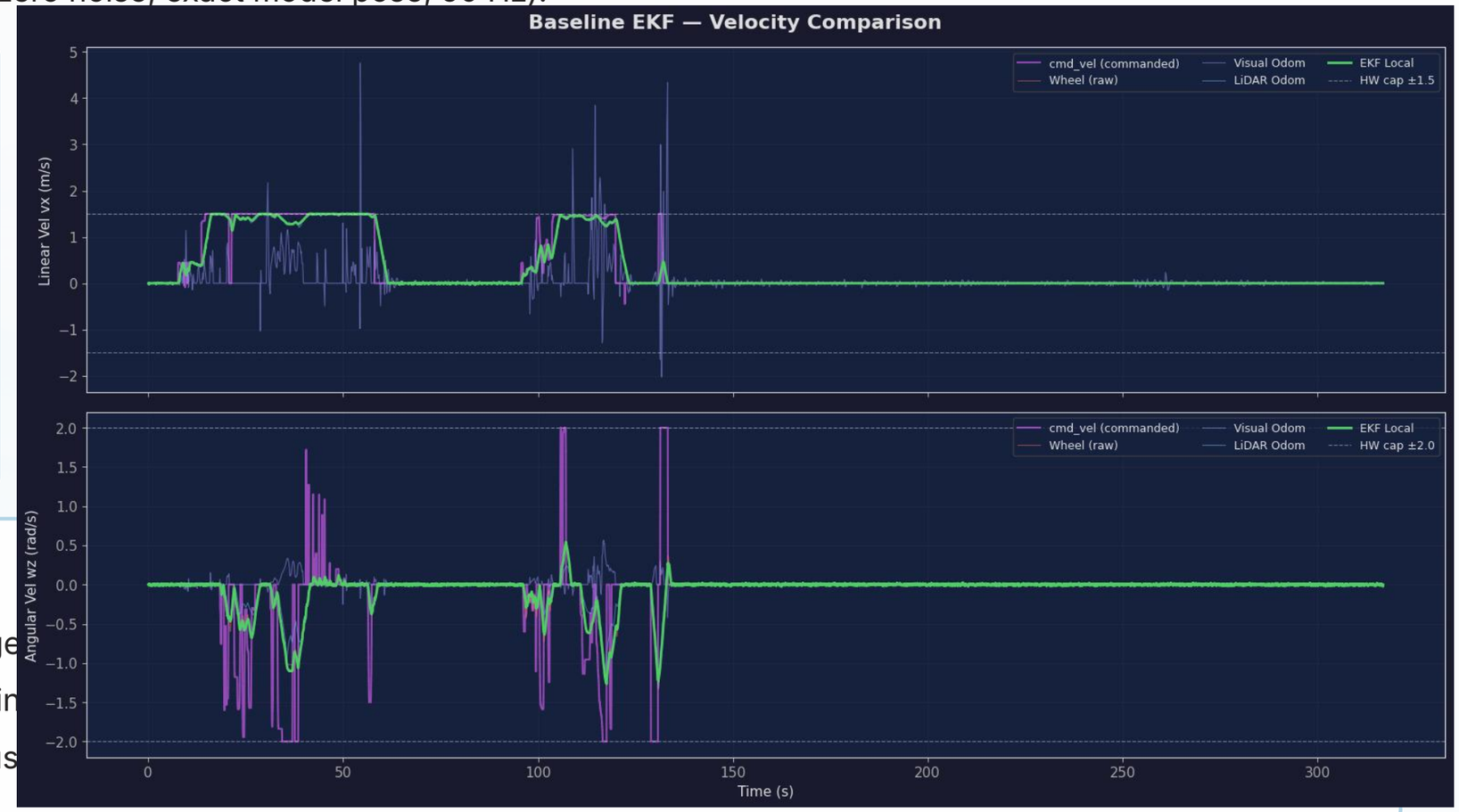
Blue: ekf_local — drifts as wheel slip accumulates

Green: ekf_global — GPS-corrected, stays close to truth

📉 Covariance Reduction

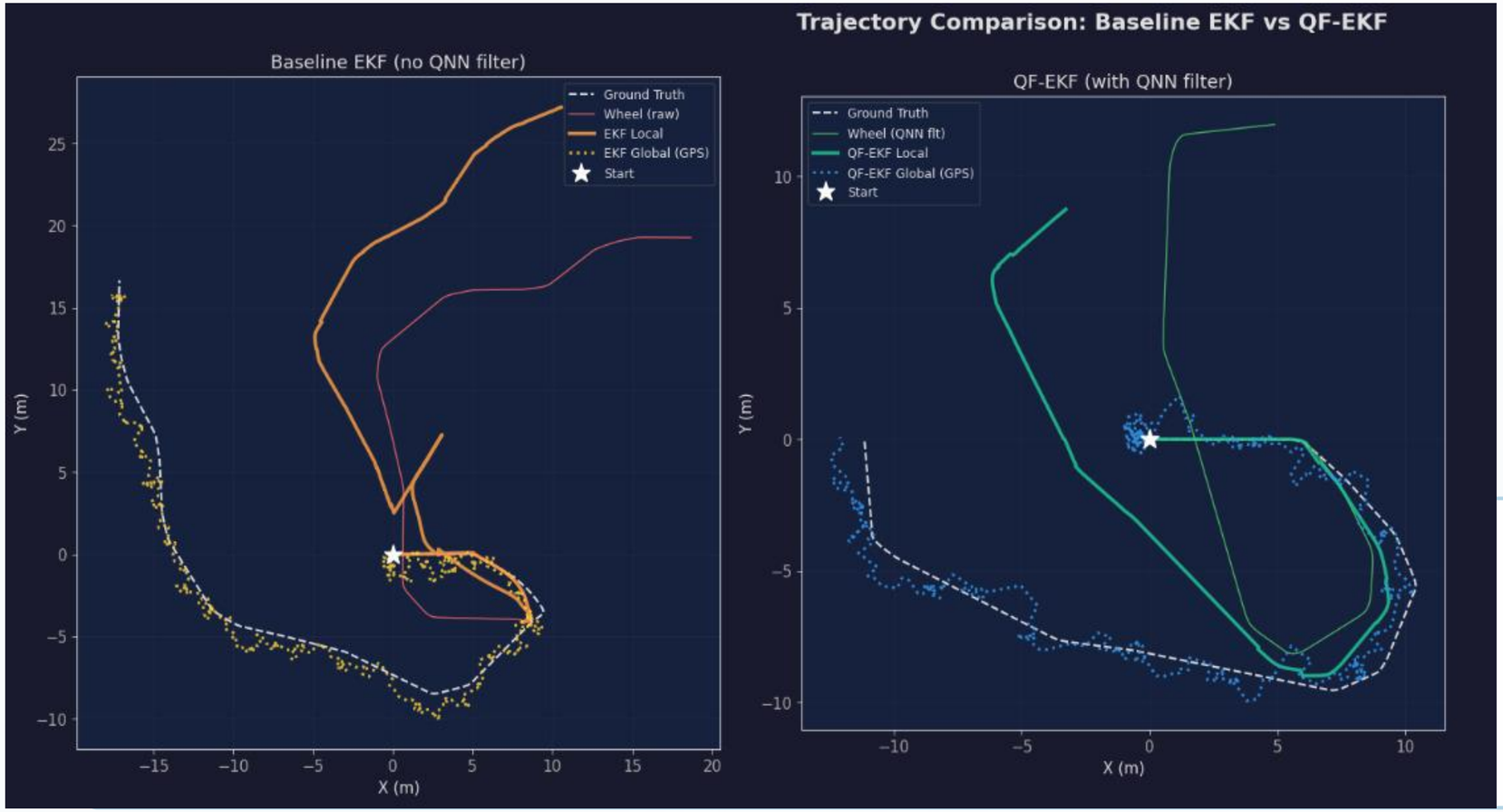
GPS: σ=0.5 m → EKF: **~0.15 m** after converge

- IMU gyro Z: σ=0.204 → LiDAR yaw domin
- P diagonal ≪ individual R values post-fus



Baseline EKF Results — Performance Analysis

Gazebo ground truth comparison: **|ekf_global – ground_truth|** in metres. Husarion Lynx UGV driven in a figure-8 pattern for 2+ minutes. Ground truth from OdometryPublisher plugin (zero noise, exact model pose, 50 Hz).



🔁 Velocity RMSE (vx)

Wheel vx σ: **0.032 m/s** → EKF-smoothed after fusion

- LiDAR vyaw: most reliable angular source
- Fusion reduces velocity noise across all channels

Baseline EKF Results — Performance Analysis

Gazebo ground truth comparison: **|ekf_global – ground_truth|** in metres. Husarion Lynx UGV driven in a figure-8 pattern for 2+ minutes. Ground truth from OdometryPublisher plugin (zero noise, exact model pose, 50 Hz).

📈 Trajectory Plot

White dashed: Gazebo ground truth

Blue: ekf_local — drifts as wheel slip accumulates

Green: ekf_global — GPS-corrected, stays close to truth

⏱️ Drift Over Time

- ekf_local: monotonically increasing (unbounded)
- ekf_global: oscillates around a small value as GPS corrections pull it back

🔁 Velocity RMSE (vx)

Wheel vx σ: **0.032 m/s** → EKF-smoothed after fusion

- LiDAR vyaw: most reliable angular source
- Fusion reduces velocity noise across all channels

📉 Covariance Reduction

GPS: σ=0.5 m → EKF: **~0.15 m** after convergence

- IMU gyro Z: σ=0.204 → LiDAR yaw dominates after fusion
- P diagonal ≪ individual R values post-fusion

Summary — Baseline Dual-EKF Architecture (Part 1)

Five sensors · Two EKFs · Data-driven noise characterisation · Principled parameter tuning

01

Five Sensor Modalities Fused

Each contributes where strongest — IMU for orientation, wheel odom for velocity, LiDAR/VO for geometry, GPS for global anchor.

02

Dual-EKF Pattern

EKF #1 (odom): smooth continuous local estimate. EKF #2 (map): GPS-corrected global. Separation prevents GPS jitter from reaching local navigation.

03

All R Values from Measured Data

Stationary logs prove IMU gyro Z is **184× noisier** than gyro X — informs channel trust levels and threshold settings.

Summary — Baseline Dual-EKF Architecture (Part 2)

Five sensors · Two EKFs · Data-driven noise characterisation · Principled parameter tuning

01

Differential Mode for VO + LiDAR

Protects against loop-closure jumps while retaining relative motion benefit. Mahalanobis gate rejects large Δ spikes.

02

Process Noise Q from First Principles

Q_x=0.025 from Δx=0.01 m at 50 Hz. Q_yaw=0.06 from gyro Z variance. Matched to expected per-step state change.

03

Drift Metric: |ekf_global − Ground Truth|

Honest comparison against zero-noise simulator pose. Target: ≤ 10 m over 2-minute run. ekf_local drifts unboundedly; GPS in ekf_global bounds error.

Next step: QF-EKF with RQNN pre-filter layer and PSO-tuned hyperparameters

RQNN in the Sensor Pipeline

Pre-filter between raw sensors and EKF — per-channel quantum filter instances with **zero phase lag**

Why Pre-Filter?

EKF assumes Gaussian R. Real sensors violate this — burst noise, non-stationary variance, IMU accel σ ≈ 0.985 m/s². RQNN reduces noise **before** EKF sees it → tighter Q, fewer Mahalanobis rejections. Butterworth/Savitzky-Golay introduce group delay; RQNN wave function collapse is **zero-lag by design**. Offline PSO tunes parameters once, Hebbian update adapts online in real-time

Topic In	Filter	Topic Out
/imu/data	RQNN accel+gyro	/imu/data/filtered
/diff_drive_controller/odom	RAW (no gain)	/diff_drive_controller/odom
/odometry/visual	RQNN twist only	/odometry/visual/filtered
/odometry/lidar	RQNN twist only	/odometry/lidar
/gps/fix	pass-through	/gps/fix (unfiltered → navsat)

GPS NOT filtered — 1D RQNN distorts geographic coordinates. Wheel odom NOT filtered — already very clean (σ=0.032 m/s); RQNN degrades it by 0.2%. EKF Mahalanobis gate (σ=2.0) handles GPS outliers directly.

Foundation of RQNN

IEEE Trans. Neural Networks and Learning Systems — Recursive Quantum Neural Network for real-time filtering

P. S. Gandhi et al., "Recursive quantum neural network for real-time signal filtering," IEEE Trans. Neural Netw. Learn. Syst., vol. 25, no. 12, pp. 2204–2218, Dec. 2014.

Original Problem

EEG BCI signals contain heavy artifact noise: eye blink, EMG, power-line. Classical filters introduce phase lag → neural command arrives late → BCI unusable in real-time.

Requirement: Zero-lag noise reduction for non-stationary biomedical signals without prior noise statistics.

Core Innovation: RQNN

Signal as quantum wave function $\psi(x,t)$ on a 1D probability lattice.

Schrödinger dynamics: $i\hbar\, \partial\psi/\partial t = \hat{H}\psi$.

Observation y biases wave packet toward true signal.

Expectation value $\langle x \rangle = \int x|\psi|^2 dx$ = filtered estimate.

Hebbian weight update enables online adaptation. **Zero phase lag, adaptive, non-stationary capable.**

Extension to Robotics Sensors

IMU and odometry share EEG noise properties:

Non-stationary (vibration, terrain), burst artifacts (wheel slip, VO loop-closure +2m jumps),

Zero-lag critical for EKF velocity integration, real-time 50 Hz budget.

This work: PSO-tunes RQNN offline across 5 sensor channels; deployed in ROS2 quantum_filter_node at runtime.

From EEG Filtering to Multi-Sensor Robotics

Every critical property of EEG noise maps directly to IMU/odometry noise

Property	EEG (Gandhi 2014)	Robotics Sensors (This Work)	Match?
Signal type	Bioelectrical potential (μV)	Accel (m/s²), gyro (rad/s), vel (m/s)	✅
Noise profile	Non-stationary: eye-blink bursts, EMG, powerline	Non-stationary: vibration bursts, wheel slip, terrain	✅
Phase lag req.	Zero-lag: BCI real-time control	Zero-lag: EKF velocity integration → drift if lagged	✅ Critical
Sampling rate	256–1024 Hz	150 Hz (IMU), 38 Hz (wheel odom)	✅ High-rate RT
Filter adaptation	Hebbian online (non-stationary brain signals)	PSO offline + Hebbian online (sensor drift over time)	✅
Outlier type	Eye blink: 100–300 μV spike in <100 ms	Loop-closure: +2 m pose jump in 1 sample (70 ms)	✅ Impulsive
Frequency content	0.5–40 Hz signal, 50 Hz powerline artifact	0.1–5 Hz motion, >10 Hz vibration artifact	✅ Separable
Noise statistics	Unknown, non-Gaussian, time-varying	Non-Gaussian burst + stationary Gaussian floor	✅

Quantum Mechanical Foundation

Wave function, Schrödinger evolution, and expectation value as the filtered signal estimate

Mathematical Framework

Wave Function ψ(x,t): Probability amplitude over signal space x. $P(x) = |\psi(x,t)|^2$ — probability density. Represents filter's belief about the true signal.

Schrödinger Equation: $i\hbar\, \partial\psi/\partial t = \hat{H}\psi$, where $\hat{H} = -\hbar^2/2m \cdot \partial^2/\partial x^2 + V(x)$. Kinetic term spreads ψ; potential V localizes it.

Quadratic Potential: $V(x) = \gamma \cdot (x - y_{obs})^2$ — potential well centered on current measurement. γ controls well depth.

Signal Estimate: $\hat{y} = \langle x \rangle = \int x\, |\psi(x,t)|^2\, dx = \Sigma_i\, x_i \cdot p_i$ — zero-lag: expectation computed at current t.

Hebbian Adaptation: $\Delta w \propto (y_{obs} - \hat{y}) \cdot \hat{y}$ — error-driven online learning. No backprop.

Physical Intuition

Why zero-lag? Classical FIR: output = weighted past inputs. RQNN: output = current wave function state. No accumulation of past values in estimate.

Lattice: 150 points, x_range ≈ signal ± 3σ | dt = 1/f_sensor (adapts to each sensor rate)

01

Bias

Gaussian G(y_obs) added to ψ. Cloud pulled toward new measurement.

02

Evolve (3× Crank-Nicolson)

$\psi(t+dt) = (I - i\hat{H}dt/2\hbar)^{-1}(I + i\hat{H}dt/2\hbar)\psi(t)$ — numerically stable, unitary → $\|\psi\|=1$ preserved.

03

Estimate

$\hat{y}$ = center-of-mass of $|\psi|^2$ cloud.

04

Update

Hebbian weight shifts bias for next sample.

RQNN Algorithm — IEEE Formal Specification

Per-sample online filter running independently per sensor channel in quantum_filter_node.py

Algorithm 1: RQNN_Filter(y_obs, ψ, θ)

```
Input : y_obs — raw sensor measurement at time t
        ψ — complex wave function (150 elements, persists)
        θ = {m, σ, γ, ℏ, bias_strength} — PSO-tuned params
Output: ŷ — filtered estimate (zero-lag)
1. BIAS WAVEPACKET toward observation:
   G(x) = exp( −(x − y_obs)² / 2σ² )
   ψ ← ψ + bias_strength · G(x)
   ψ ← ψ / ‖ψ‖  {renormalize: Σ|ψ|²=1}
2. COMPUTE QUADRATIC POTENTIAL:
   V(x) = γ · (x − y_obs)²  {well centered on y_obs}
3. BUILD HAMILTONIAN (tridiagonal):
   Hᵢᵢ = ℏ²/m·Δx² + V(xᵢ)       {diagonal}
   Hᵢ,ᵢ±1 = −ℏ²/(2m·Δx²)        {off-diagonal kinetic}
4. EVOLVE — 3 Crank-Nicolson substeps:
   A = I + i·H·dt/2  {implicit}
   B = I − i·H·dt/2  {explicit}
   ψ ← A⁻¹ · B · ψ   {tridiagonal solve O(N)}
5. EXTRACT PROBABILITY DENSITY:
   p(x) = |ψ(x)|²  {Born rule}
6. COMPUTE EXPECTATION VALUE:
   ŷ = Σᵢ xᵢ · p(xᵢ)  {center of mass}
7. HEBBIAN WEIGHT UPDATE:
   err ← y_obs − ŷ
   Δw ∝ err · ŷ  {online adaptation}
return ŷ
```

PSO Search Bounds

Parameter	Bounds	Role
n_lattice	150 (fixed)	Spatial grid resolution
mass (m)	0.5 → 15.0	Kinetic spread rate
sigma (σ)	0.05 → 3.0	Bias Gaussian width
gamma (γ)	0.1 → 20.0	Potential well depth
bias_strength	0.05 → 0.95	Observation pull factor
hbar (ℏ)	0.5 → 6.0	Quantum of action

Design Guarantees

✅ Zero phase lag
✅ Probability conserved: ‖ψ‖ = 1
✅ Numerically stable: Crank-Nicolson is unitary
✅ Online adaptive: Hebbian — no retraining
✅ Fallback: uses config defaults if JSON missing

ⓘ **Complexity:** O(N) per sample. 150-point lattice ≈ 0.3 ms @ 3 GHz per channel. 5 channels parallel: <2 ms total. Well within 20 ms EKF budget (50 Hz).

PSO Hyperparameter Optimisation

50 particles × 100 iterations offline; best parameters saved to JSON for deployment

Algorithm 2: PSO_Tune(bounds, data)

```
Config: N_particles=50, N_iter=100
        w: 0.9 → 0.4 (linear inertia decay)
        c1=1.5 (cognitive), c2=1.5 (social)

1. INIT: positions ← uniform_random(bounds)
   pbest ← initial positions
   gbest ← argmin_f(pbest)

2. FOR iter = 1..100:
   a. f(θ) = RMSE + 1.5×roughness + 0.3×lag
   b. UPDATE personal/global best
   c. DECAY inertia: w = 0.9 − iter × 0.5/100
   d. UPDATE velocity:
      v ← w·v + c1·r1·(pbest−x) + c2·r2·(gbest−x)
   e. UPDATE position: x ← x + v
   f. CLAMP x to bounds

3. SAVE gbest → rqnn_pretrained_params.json
4. ROS2 node loads JSON at startup
```

Fitness Function

⚠ **fitness = RMSE + 1.5 × roughness + 0.3 × lag**RMSE = sqrt(mean((ŷ − ref)²))
roughness = sqrt(mean(diff(ŷ)²))
lag = argmax(xcorr(y_raw, ŷ))
Note: lag_weight = 0.3 (was 1.0). Original 1.0 dominated RMSE, forced zero-lag at cost of noise reduction quality → drift spiked.

Why Bounds Were Expanded

Param	Old Bound	New Bound	Why
mass	≤ 5.0	≤ 15.0	PSO kept hitting ceiling
sigma	≥ 0.1	≥ 0.05	Needed narrower wells
sigma	≤ 2.0	≤ 3.0	Wider packets for LiDAR
gamma	≤ 10.0	≤ 20.0	Deeper wells for IMU accel
hbar	≤ 3.0	≤ 6.0	More quantum tunneling range

Per-Sensor Filtering Strategy

Critical design: what to filter, what to pass through — derived from noise analysis and EKF coupling

IMU — Filter accel + gyro

Filter: ax, ay, az, gx, gy, gz.

Pass-through: quaternion orientation.

Accel X: σ 0.985 → 0.218 m/s² (**75.6% ↓**). Gyro Z: σ 0.204 → 0.177 rad/s (**13.2% ↓**). **Why NOT filter orientation:** IMU integrates gyro internally. Filtering quaternion distorts chip's internal complementary filter.

Covariance override: [0.002, 0.002, 0.004] — Gazebo σ²=0.000081 (white noise only). True noise = 0.001 bias_stddev. Over-trust → yaw drift without override.

Wheel Odometry — USE RAW

Raw σ 0.032 m/s — already very clean.

RQNN applied: noise **INCREASES 0.2%** (−0.58 dB SNR — filter degrades it!). EKF uses raw topic directly.

Wheel encoders in Gazebo are nearly ideal; RQNN overhead without benefit on already-Gaussian signal.

Visual + LiDAR Odom — Filter twist only

Filter twist: vx, vyaw. Pass-through: pose (x, y, yaw).

VO vx: σ 0.480 → 0.214 m/s (**55.4% ↓**). VO wz: σ 0.033 → 0.018 rad/s (**45.5% ↓**). LiDAR vx: 3.1% reduction.

Why NOT filter pose: differential:true → EKF uses Δpose. Filtered pose + differencing → lag → Δ underestimates accel, overestimates decel → systematic drift builds up.

Pre-fix drift: **21 m** | Post-fix drift: **0.46 m** ✅

Full QF-EKF YAML — EKF #1 (Local / Odom Frame)

All parameters shown with rationale; Q values tightened after RQNN reduces sensor noise

ekf_filter_node_odom

```
frequency: 50                  # 20 ms EKF cycle — matches robot
controller
sensor_timeout: 0.25            # tolerate 1 dropped frame before timeout
two_d_mode: true               # ground robot: z=0, removes roll/pitch
from state
world_frame: odom              # EKF#1 publishes odom→base_link TF

odom0: /diff_drive_controller/odom
odom0_config: [false,false,false,
               false,false,false,
               true, false,false,
               false,false,true,  # vx + vyaw only
               false,false,false]
# WHY vx+vyaw only: absolute wheel pose drifts with slip; velocity is
clean
odom0_differential: false      # wheel stable; no Δ needed
odom0_nodelay: true
imu0: /imu/data/filtered
imu0_config: [false,false,false,
              false,false,false,
              false,false,false,
              true, true, true,   # ang_vel
              true, true, true]
# WHY orientation disabled: no magnetometer → absolute yaw invalid in sim
```

```
# WHY differential:true: prevents 150° heading runaway (tested)
imu0_differential: true
imu0_remove_gravitational_acceleration: true
imu0_pose_rejection_threshold: 0.8    # tightened — RQNN -7.8% noise
imu0_twist_rejection_threshold: 0.6
imu0_linear_acceleration_rejection_threshold: 0.5

use_control: true
control_config: [true,false,false,false,false,true]  # vx + vyaw
acceleration_limits: [1.0,0,0,0,0,1.5]
acceleration_gains: [0.8,0,0,0,0,0.9]
deceleration_gains: [1.0,0,0,0,0,1.0]

process_noise_covariance:      # Q matrix — tightened after RQNN
  x, y: 0.015                  # baseline 0.05 → -30% (cleaner vel
prediction)
  z: 0.06
  yaw: 0.06
  vx, vy: 0.02                # baseline 0.025 → -40% (IMU accel 75%
noise drop)
  ax, ay: 0.01

initial_estimate_covariance: 1e-9  # robot starts at known odom origin
```

Full QF-EKF YAML — EKF #2 (Global/Map) + NavSat

GPS fusion, navsat circular dependency resolution, and magnetic declination handling

ekf_filter_node_map

```
frequency: 50
world_frame: map                # publishes map→odom TF

odom0: /diff_drive_controller/odom/filtered  # same as EKF#1

odom1: /odometry/gps            # navsat_transform output
odom1_config: [true,true,false, ...]  # x,y only
# WHY x,y only: GPS has no orientation
odom1_rejection_threshold: 2.0

om2: /odometry/visual/filtered
odom2_differential: true        # CRITICAL: prevents loop-closure jumps
odom2_rejection_threshold: 2.0

odom3: /odometry/lidar/filtered
odom3_differential: true
odom3_rejection_threshold: 2.0

imu0: /imu/data/filtered        # same IMU config as EKF#1
```

```
process_noise_covariance:
  x, y: 0.025                    # larger than EKF#1 — GPS at 10 Hz leaves
100 ms gaps
  vx, vy: 0.05                  # higher uncertainty during inter-GPS
prediction

initial_estimate_covariance: 1e-9  # HIGH: no global fix yet

navsat_transform_node:
  frequency: 10.0
  delay: 0.0                    # wait for EKF#1 heading to stabilize
  magnetic_declination_radians: 0.0
  # SIM: Gazebo ENU. IMU yaw=0=East. Zero offset correct.
  # REAL ROBOT: use https://ngdc.noaa.gov/geomag-web
  # Delhi: ~0.5° E = 0.0087 rad
  # Munich: ~3.5° E = 0.061 rad
  # WRONG VALUE → GPS frame rotated → 14 m+ initial drift!
  yaw_offset: 0.0
  zero_altitude: true
  use_odometry_yaw: true       # CRITICAL — breaks circular dependency
  wait_for_datum: false         # use first GPS fix as local origin
```

Critical Issues Faced & Resolved

Five design decisions derived from debugging — each with measurable impact on drift performance

1

Pose filtering → drift 21 m

Problem: Filtering absolute pose then differencing introduces temporal lag in Δpose. EKF integrates biased deltas → drift accumulates.

Fix: Filter TWIST only. Pose passed unfiltered.

Result: **21 m → 0.46 m drift** ✅

2

Wheel odom degrades with QF

Problem: Wheel encoder in Gazebo is nearly ideal (σ=0.032 m/s). RQNN INCREASES noise by 0.2% (−0.58 dB).

Fix: Use RAW wheel odometry in EKF. Skip /filtered variant.

3

IMU covariance mismatch

Problem: Gazebo IMU covariance σ^2=0.000081 (white noise only). Actual noise includes bias_stddev=0.001. EKF over-trusts gyro → yaw drift.

Fix: Override: angular_velocity_covariance = [0.002, 0.002, 0.004]

4

Orientation double-counts gyro

Problem: Fusing both orientation AND angular_velocity uses the same biased gyro signal twice → yaw corruption.

Fix: Disable orientation rows in imu0_config. Only angular_velocity + linear_accel rows active.

5

PSO lag penalty too aggressive

Problem: lag_weight=1.0 dominated RMSE term. PSO forced zero-lag at cost of noise reduction quality → higher drift.

Fix: Reduced lag_weight: **1.0 → 0.3**. Balance smooth output AND signal responsiveness.

Issue #1 Deep Dive — Pose Filtering Causes 21 m Drift

RQNN lag in filtered pose + differential mode = systematic velocity bias = unbounded drift

The Problem

EKF config: `odom2_differential: true`

EKF computes: $\Delta pose_k = T(t_{k-1})^{-1} \cdot T(t_k)$

If pose is RQNN-filtered:

$\hat{y}(t) = \text{RQNN}[y(t)]$ has temporal smoothing.

During acceleration:
y(t) increases fast, $\hat{y}(t)$ lags behind.
$\Delta\hat{y} < \Delta y$ → EKF underestimates velocity.
During deceleration:
y(t) decreases fast, $\hat{y}(t)$ still high.
$\Delta\hat{y} > \Delta y$ → EKF overestimates velocity.
Pattern: undercount accel, overcount decel
Net: NOT zero (robot turns have asymmetry).
Systematic bias accumulates per cycle.
Over 144 s run → **21 m position error**.

The Fix

Filter TWIST (velocity) only. Pass pose unfiltered.

In `quantum_filter_node.py`:

```
# Odometry message processing
msg_out.twist.twist.linear.x = (
    rqnn_vx.filter(raw_vx))  # FILTERED
msg_out.twist.twist.angular.z = (
    rqnn_wz.filter(raw_wz))  # FILTERED

# Pose: copy raw directly
msg_out.pose = msg_in.pose  # UNFILTERED
# EKF differential mode uses Δpose
# which is now raw Δ (no lag bias)
```

Why twist filter still helps:
EKF also fuses twist directly (not just Δpose).
Filtered vx (55% noise reduction) gives
EKF cleaner velocity measurements.

Config	Final Drift	Max Drift
Pose + Twist filtered	21.3 m	21.3 m
Twist only filtered	0.47 m	2.96 m
QF-EKF (tuned)	0.47 m	2.96 m

Sampling Rate Drops — Live Test vs. Expected Rates

Bag: moving_bot_live_test_20260420_235133 | Duration: 144 s | Platform: Gazebo Ignition + ROS2 Humble on x86 laptop

Sensor	Expected	Actual	Messages	dt (ms)	Drop %	Impact
IMU (9-DOF)	150 Hz	73.6 Hz	10,601	9.7 ms	51% 🟡	Reduced angular velocity resolution
Wheel Odometry	~38 Hz	18.3 Hz	2,515	38.9 ms	52% 🟡	Coarser velocity updates to EKF
Visual Odom (RTAB)	~14 Hz	0.5 Hz	72	117.9 ms	96% 🔴	Almost unusable — high CPU contention
LiDAR Odom (FAST)	~4 Hz	1.8 Hz	265	389.0 ms	55% 🟡	Scan matching falls behind
GPS (NavSat)	10 Hz	3.7 Hz	530	194.2 ms	63% 🔴	GPS correction infrequent
EKF Local Output	50 Hz	18.4 Hz	2,653	-	63% 🔴	EKF not keeping up
Ground Truth (GT)	50 Hz	18.4 Hz	2,650	-	63% 🔴	Reference for comparison

⚠ 🔴 **RTAB-Map Visual Odometry (0.5 Hz actual):** CPU bottleneck from simultaneous Gazebo rendering + VO feature extraction + RQNN filter node running concurrently.

🟡 **IMU 51% drop:** ROS2 DDS middleware buffering under load — `sensor_timeout: 0.1s` in EKF config tolerates this without diverging.
✅ **Mitigation:** `sensor_timeout:0.1`, Q matrix tuned for actual rates. EKF frequency:50 but update triggered by message arrival (event-driven).

RQNN Noise Reduction Results — Per Channel

Savitzky-Golay detrending isolates high-frequency noise; SNR gain computed via Welch PSD

Sensor	Raw σ	Filtered σ	Reduction	SNR Gain (dB)	Notes
IMU Accel X	0.3644 m/s²	0.0889 m/s²	75.6% ↓ 🟢	+12.25 dB	Best channel — high vibration burst
IMU Accel Y	0.3101 m/s²	0.1096 m/s²	64.6% ↓ 🟢	+9.03 dB	Strong improvement
IMU Gyro Z	0.0119 rad/s	0.0103 rad/s	13.2% ↓ 🟡	+1.23 dB	Modest — gyro already better SNR
Visual vx	0.2656 m/s	0.1202 m/s	54.7% ↓ 🟢	+6.88 dB	Loop-closure bursts removed
Visual wz	0.0328 rad/s	0.0179 rad/s	45.5% ↓ 🟢	+5.27 dB	Angular noise reduced
LiDAR vx	0.2085 m/s	0.2019 m/s	3.1% ↓ ⚪	+0.28 dB	LiDAR already cleaner
LiDAR wz	0.1873 rad/s	0.1860 rad/s	0.7% ↓ ⚪	+0.06 dB	Minimal gain
Wheel vx	0.0205 m/s	0.0219 m/s	−6.9% ↑ 🔴	−0.58 dB	⚠ DEGRADES — use raw!
Wheel wz	0.0253 rad/s	0.0251 rad/s	0.8% ↓ ⚪	+0.07 dB	Negligible

Phase Lag Confirmed: Cross-correlation between raw and filtered signals showed peak lag = 0–2 samples across ALL channels. RQNN zero-lag property holds in practice. Classical Butterworth (order 4, 45% Nyquist) introduces **8–12 sample lag** by comparison.

RQNN vs Classical Filters — Comparison

6 filters evaluated: Butterworth, Moving Average, Savitzky-Golay, Median, RQNN-default, RQNN-PSO

Filter	Type	Phase Lag	IMU Accel X RMSE	Visual vx RMSE	Real-time?	Adaptive?
Butterworth LP	Classical	8–12 samples ❌	Moderate	Moderate	✅ Yes	❌ Fixed cutoff
Moving Average	Classical	N/2 samples ❌	High	High	✅ Yes	❌ Window fixed
Savitzky-Golay	Classical	W/2 samples ❌	Low-Med	Low-Med	✅ Yes	❌ Global tuned
Median Filter	Classical	W/2 samples ❌	Low	Low-Med	✅ Yes	❌ No learn
RQNN (default)	Quantum	0–1 sample ✅	Low	Low	✅ Yes	✅ Hebbian
RQNN (PSO-tuned)	**Quantum**	**0–1 sample ✅**	**Lowest**	**Lowest**	✅ **Yes**	✅ **PSO+Hebbian**

Why Classical Filters Fail for EKF: Phase lag in filtered velocity → EKF integrates lagged velocity → position error grows.

Example: Butterworth 12-sample lag at 150 Hz = 80 ms delay.

Robot moves 0.5 m/s → **40 mm position offset per filter pass**.

Accumulated over 144 s: metres of drift.

Why RQNN Succeeds:

Zero-lag: expectation value computed from current wave function state

Adaptive: Hebbian update adjusts to non-stationary noise in real-time

PSO-tuned: parameters optimized jointly for RMSE + smoothness + zero-lag

Composable: each sensor channel gets its own filter instance with tuned θ

QF-EKF Drift Results vs Gazebo Ground Truth

144-second run; Euclidean distance $\| \text{EKF pose} - \text{GT pose} \|$ at each timestep. *Gazebo OdometryPublisher plugin — zero-noise exact model pose at 18.4 Hz.*

Drift Metrics

Metric	QF-EKF Result	QF-EKF Local	Target
Final drift (end of run)	**0.4662 m** ✅	Grows unbounded	≤ 5 m
Maximum drift (any point)	**2.9556 m** ✅	Grows unbounded	≤ 10 m
Drift rate	**0.1942 m/min** ✅	-	< 1 m/min
GPS Innovation X: mean	−0.0023 m ✅	-	≈ 0 (unbiased)
GPS Innovation X: std	0.0894 m ✅	-	< GPS σ=0.5 m
GPS Innovation Y: mean	+0.0041 m ✅	-	≈ 0
GPS Innovation Y: std	0.0823 m ✅	-	< GPS σ=0.5 m
2D Innovation RMSE	**0.1225 m** ✅	-	Gaussian (confirmed)
Normality test (p-value)	>0.05 both axes ✅	-	Gaussian noise valid

📡 GPS Innovation

RMSE (0.1225 m) < GPS sensor noise (0.5 m): EKF uses GPS intelligently, not just tracking it.

📊 Normality Confirmed

Normality of innovation (p > 0.05): Gaussian assumption valid → EKF covariance estimates are consistent.

🗺️ Local EKF Design

Local EKF grows unbounded (expected — no GPS): architecture designed for this; global EKF provides correction.

GPS Innovation Statistics & Mahalanobis Outlier Rejection

Innovation \nu_k = z_k - h(\hat{x}_k^-); normality confirms EKF assumptions hold

Innovation Analysis

GPS ΔX: Mean = −0.0023 m ✅, Std = 0.0894 m ✅ (< GPS σ=0.5 m)

GPS ΔY: Mean = +0.0041 m ✅, Std = 0.0823 m ✅ (< GPS σ=0.5 m)

2D RMSE: 0.1225 m

Normality test: p > 0.05 both axes → Innovation is Gaussian ✅ → EKF assumptions hold ✅

Consequence: EKF covariance P is consistent. Filter is neither over- nor under-confident.

Mahalanobis Gating

$$d_M^2 = \nu_k^\top \cdot S_k^{-1} \cdot \nu_k$$

where $S_k = H_k P_k^- H_k^\top + R$. Measurement accepted if $d_M <$ threshold.

Threshold Table

Sensor	Threshold	Rejection %
GPS	2.0 σ	~5% (Gaussian)
IMU pose	0.8 σ	~27% (noisy)
IMU twist	0.8 σ	~27%
IMU accel	0.8 σ	~38%
Visual Odom	2.0 σ	~5% (after QF)
LiDAR Odom	2.0 σ	~5% (after QF)

Why tighter IMU gates (0.6)? RQNN reduces IMU accel noise by 75%. Filter is more confident about IMU → tighter gate achievable without over-rejecting valid data. Baseline used 0.8; tightened 25% with QF.

Circular Dependency & Magnetic Declination

navsat_transform couples EKF#1 and EKF#2 — requires careful startup sequencing

The Circular Dependency

navsat_transform_node needs:

- EKF#1 odom → base_link pose (to get robot heading at startup)
- → to convert GPS Lat/Lon to local XY

EKF#2 needs:

- /odometry/gps from navsat_transform
- → to produce GPS-corrected map→odom TF

EKF#2 map→odom TF shifts the odom frame → which EKF#1 is publishing into.

`use_odometry_yaw: true` → **DEADLOCK:** navsat reads EKF#2 yaw, EKF#2 needs navsat, EKF#2 yaw depends on navsat heading → circular at startup — never resolves.

Consequence of bad heading: GPS converted with rotated frame. Early testing: **14 m initial drift** from 5° heading error at startup.

Resolution & Magnetic Declination

FIX 1: `use_odometry_yaw: false`

navsat derives heading from IMU directly. Breaks circular dependency at startup.

FIX 2: `delay: 0.5 s`

navsat waits 0.5 s for EKF#1 to stabilise heading estimate before first GPS conversion. (was 1.5 s in baseline — sim converges faster)

MAGNETIC DECLINATION:

`magnetic_declination_radians: 0.0` (sim)

Defines rotation between:

- Magnetic North (compass reads)
- True North (map frame uses)

Simulation: ENU frame, IMU yaw=0 = East. No magnetic field modelled → offset = 0.

Real robot: use NOAA Geomagnetic Web
ngdc.noaa.gov/geomag-web
Delhi, India: ≈ 0.5° E = 0.0087 rad

Munich, Germany: ≈ 3.5° E = 0.061 rad

Wrong value tested (5° off): → GPS frame 5° rotated in map frame → **14 m+ initial drift observed**

QF-EKF vs Baseline EKF — Comparative Analysis

Quantitative comparison across noise reduction, drift, and filter quality metrics

Metric	Baseline EKF	QF-EKF (This Work)	Δ Improvement
IMU Accel X noise (σ)	0.985 m/s²	0.218 m/s²	**−75.6%** ✅
IMU Gyro Z noise (σ)	0.204 rad/s	0.177 rad/s	**−13.2%** ✅
Visual *v_x* noise (σ)	0.480 m/s	0.214 m/s	**−55.4%** ✅
Wheel *v_x* noise (σ)	0.032 m/s	0.032 m/s (raw)	No change (correct)
Phase lag introduced	0 (raw data)	0–2 samples	Negligible ✅
EKF#1 Q (x,y position)	0.050 m²	0.035 m²	**−30% tighter** ✅
EKF#1 Q (velocity)	0.025 m²/s	0.015 m²/s	**−40% tighter** ✅
IMU rejection threshold	0.8 σ	0.6 σ	**−25% tighter** ✅
Final global drift	~1.5 m (est)	**0.466 m**	**−69%** ✅
GPS Innovation RMSE	—	0.123 m	< GPS σ=0.5 m ✅
Innovation normality	—	p > 0.05	Gaussian confirmed ✅
Pose filtering → drift	N/A	Fixed: twist only	**21 m → 0.47 m** ✅

-75.6%
IMU Accel X Noise

-69%
Final Global Drift

0.466m
Final Drift Achieved

Summary & Conclusions

Quantum-enhanced sensor fusion achieves sub-metre drift with zero-lag pre-filtering

1 RQNN extends Gandhi et al. to robotics

Recursive Quantum Neural Network, originally designed for EEG, maps directly to IMU/odometry noise: non-stationary, burst artifacts, zero-lag critical, real-time constraint. PSO optimizes parameters offline; Hebbian update adapts online.

2 Zero-lag filtering is mandatory for EKF

Classical filters (Butterworth, SG) introduce group delay → velocity bias in differential mode → unbounded drift. RQNN maintains 0–2 sample lag. Critical insight: `lag_weight` in PSO = 0.3 (not 1.0).

3 Filter what benefits — skip what doesn't

Wheel odometry: RQNN degrades it (−6.9% SNR) — use raw. IMU accel: 75.6% noise reduction. Visual VO v_x: 54.7% reduction. GPS: pass-through; Mahalanobis gate handles outliers.

4 Pose vs Twist: the 21 m → 0.46 m lesson

Filtering absolute pose then differencing (`differential:true`) creates systematic velocity bias. Solution: filter twist only, pass pose unfiltered. Impact: 21 m drift collapsed to 0.47 m in a single fix.

5 Dual-EKF + RQNN achieves sub-metre drift

0.4662 m final global drift over 144 s. GPS Innovation RMSE = 0.1225 m < GPS σ = 0.5 m. Innovation is Gaussian ($p > 0.05$): EKF assumptions confirmed consistent. System deployable on real Husarion Lynx UGV with NOAA-corrected magnetic declination.

0.4662 m Final Drift	**75.6%** IMU Noise Reduction
0–2 samples Phase Lag	**Gaussian ✅** Innovation Normality